

Self-Defeating Audits: Reversible Auditor-Blinding and Attribution Poisoning by a Low-Privilege Role in Trigger-Based Database Auditing

Thamilvendhan Munirathinam

Abstract

In-database audit trails are widely treated as ground truth for forensic reconstruction and compliance. We revisit that assumption for the most widely-copied trigger-based auditing pattern in PostgreSQL — an `AFTER` row trigger calling a `SECURITY DEFINER` function that serialises each write via an unqualified `row_to_json` into a privileged audit table — and ask a question the existing anti-forensics literature does not: *can the auditee itself, holding only the application's privileges (as reached through SQL injection, with no superuser and no OS access), reversibly blind this auditor and poison attribution from within the database?* We build a disposable, synthetic-data lab and work a feasibility matrix cell-by-cell on stock PostgreSQL 14 and 16. We find that plain `search_path` shadowing of a catalog built-in fails (as expected, since `pg_catalog` is resolved first), but planting an *exact composite-type overload* of `row_to_json` in a writable `public` schema silently nulls captured rows against an unpinned `SECURITY DEFINER` trigger — reachable on **stock PostgreSQL ≤14**, where `PUBLIC` holds `CREATE` on `public` by default, and misconfiguration-gated on 15+. Where the audit function is owned by the application's own role, the attacker replaces its body to drop writes and then restores the original source *byte-for-byte* (function source is stored verbatim in `pg_proc`), leaving a clean object diff — while an independent write-versus-audit canary and a DDL event-trigger still reveal the gap. We further demonstrate a `current_setting`-gated dormant bypass and, under a broad `INSERT` grant, fabricated back-dated audit rows indistinguishable from genuine ones using the audit table alone. The individual primitives are known — CVE-2018-1058 `search_path` shadowing and the `SECURITY DEFINER` escalation class — and we cite them as such; our contribution is their composition into a *self-defeating-audit* threat model at **application privilege**, an executed defense evaluation, and a reproducible artifact. We show that pinning the trigger's `search_path`, revoking `CREATE` on `public`, correcting audit-object ownership, and — decisively — engine-level logging (`pgaudit / log_statement`) combined with a hash-chained append-only sink each stop a specific subset, and that only engine-level plus out-of-engine tamper-evidence closes the whole set. We further confirm the threat is a cross-engine class on the MySQL family (MariaDB 10.6/10.11) and SQL Server (2025/v17): the name-resolution shadowing class does not transfer (neither has `search_path`), but trigger-replacement, dormant bypass, and attribution poisoning do. MySQL's `DEFINER`-privilege binding makes blinding more detectable (the attacker cannot restore the original trigger), whereas SQL Server, like PostgreSQL, allows byte-perfect restore and additionally exposes a zero-residue `DISABLE TRIGGER` primitive that its own DDL-trigger defense does not catch.

1. Introduction

Database audit trails underpin incident response, insider-threat investigation, and regulatory compliance. A pervasive folk design — reproduced across countless deployments from the PostgreSQL wiki "audit trigger" recipe — captures every write with a row-level trigger that serialises the changed row into an append-oriented audit table. Its appeal is that it lives *inside* the database, close to the data, and needs no external infrastructure.

The security literature on defeating such trails is almost entirely written from the perspective of a **privileged** adversary: a database administrator, a compromised superuser, or an insider who has

escalated to administrative rights and simply disables logging or edits the log files [MemTraceDB]. That threat model is important but incomplete. The most common way an application database is actually attacked is **SQL injection**, which grants the attacker exactly the *application role's* privileges — no more. The question this paper asks is whether that low-privilege foothold is already sufficient to subvert the in-database auditor that is supposed to be recording the attack.

We answer empirically, with a reproducible lab, and we are careful to separate what is already known from what is new. Concretely, we contribute:

1. **A threat-model inversion.** We evaluate reversible auditor-blinding and attribution poisoning by the *auditee* at **application privilege** against an *unaudited* auditor — a setting the privileged-adversary anti-forensics literature does not cover.
2. **An empirical feasibility matrix** on stock PostgreSQL 14 and 16, distinguishing what blinds a write from what does not, and — critically — under exactly which version and `public`-schema ACL each result holds. We show the "structural on stock defaults" outcome survives only for PostgreSQL ≤ 14 .
3. **A precise reversibility result.** Because `pg_proc.prosrc` stores function source verbatim, a naive semantically-equivalent restore is detectable, but a capture-and-replay restore is byte-identical and defeats object-diffing entirely — which we use to argue that diffing audit *code* is insufficient on its own.
4. **Two under-explored composition elements:** a `current_setting`-gated dormant trigger bypass and in-database attribution poisoning that is indistinguishable using the audit table alone.
5. **An executed defense evaluation** (a defense $\times$ attack matrix), not an asserted one, including a hash-chained append-only sink and engine-level logging as the closing controls.

We deliberately do not claim the underlying primitives as novel; they are CVE-2018-1058 and the `SECURITY DEFINER search_path` escalation class, and we position against them throughout.

2. Threat model

The adversary is the **application role**, reached via SQL injection into an application that connects to PostgreSQL as that role. The adversary can therefore issue `SELECT/INSERT/UPDATE/DELETE` against the business tables the application uses, plus any DDL the application role is (mis)granted. The adversary has:

- **no superuser**, and cannot become one;
- **no operating-system access** (cannot touch log files, `postgresql.conf`, or the data directory);
- **no rights over the audit schema or its objects**, except where a specific experiment models a common operator misconfiguration (and we say so explicitly).

The defender is a database administrator who set up trigger-based auditing following common guidance and who inspects the database for tampering (object diffs, row counts) but does not necessarily run engine-level or out-of-engine logging. The security goal is audit **integrity** and **attribution**: every audited write is recorded, and recorded actor/time metadata is trustworthy.

3. Background and related work

3.1 `search_path` and CVE-2018-1058

PostgreSQL resolves unqualified names against the schemas on `search_path`, with `pg_catalog` searched implicitly first unless placed explicitly. CVE-2018-1058 [PGSEC2018; PGCOMMIT2018; PGWIKI2018]

established that "adding a schema to `search_path` effectively trusts all users having `CREATE` privilege on that schema," because a like-named object planted in an earlier schema can shadow a built-in. The canonical mitigations are to schema-qualify calls (e.g. `pg_catalog.row_to_json`), pin `search_path` (including to the empty string via `set_config`), and `REVOKE CREATE ON SCHEMA public FROM PUBLIC` — the last of which became the default ACL in PostgreSQL 15. Our vectors 1A/1B are instances of this class; we cite it as prior art and measure exactly when it does and does not yield audit-blinding.

3.2 SECURITY DEFINER functions

A `SECURITY DEFINER` function executes with the privileges of its owner. This is *necessary* for a trigger-based auditor: a low-privilege writer must be able to cause an insert into a privileged audit table, which only works if the trigger function runs as a role that can write it. The same feature is a well-documented escalation surface: an unpinned `SECURITY DEFINER` function that resolves an unqualified function or operator to an attacker-planted object runs that object with the owner's privileges [CYBERTEC; PGCOMMIT2018]. Our victim is faithful to the wiki pattern precisely because it is `SECURITY DEFINER`.

3.3 Database anti-forensics assumes a privileged adversary

Prior work on defeating database audit and forensic reconstruction consistently assumes an administrative adversary. MemTraceDB [MemTraceDB] states its attacker "has gained privileged access ... a malicious insider or an external threat actor who has escalated their credentials to an administrative level," and blinds by disabling logging or editing log files. Practitioner guidance on trigger-based auditing likewise frames its weaknesses as operational burden and, at most, superuser/owner trigger-disabling [CYBERTEC-ROW]. We are not aware of prior work evaluating reversible blinding of an unaudited trigger-based auditor by a low-privilege, injection-only role.

3.4 Tamper-evident logging

That an unaudited log is tamperable is textbook: a tamper-evident log "requires frequent auditing" by "at least one auditor ... assumed to be incorruptible," and if the logger knows an event will not be audited, tampering is undetectable [CROSBY2009]. Append-only storage is necessary but not sufficient without cryptographic tamper-evidence (hash-chaining or signatures) [SCHNEIER1998; RFC6962]. This grounds both our reversibility analysis and the hash-chained-sink defense.

4. The victim

We build the representative victim as the DBA. A business schema `app` owned by the application role holds `app.accounts`. A separate `audit` schema owned by the DBA holds `audit.logged_actions(event_id, table_name, action, row_data jsonb, changed_by, client_ip, logged_at)`. The trigger function is:

```
CREATE FUNCTION audit.if_modified() RETURNS trigger AS $$
DECLARE payload jsonb;
BEGIN
  IF (TG_OP = 'DELETE') THEN payload := row_to_json(OLD)::jsonb;
  ELSE payload := row_to_json(NEW)::jsonb; END IF;
  INSERT INTO audit.logged_actions(table_name, action, row_data)
  VALUES (TG_TABLE_NAME, left(TG_OP,1), payload);
  RETURN COALESCE(NEW, OLD);
END; $$ LANGUAGE plpgsql SECURITY DEFINER;
```

fired `AFTER INSERT OR UPDATE OR DELETE ... FOR EACH ROW`. The application role is granted only `USAGE` on `app` and `SELECT/INSERT/UPDATE/DELETE` on `app.accounts`. We note one faithful correction to the naive wiki snippet: the function **must** be `SECURITY DEFINER`, otherwise a low-privilege writer's own trigger-driven

insert into the audit table fails with a permission error rather than auditing — and it is exactly this `SECURITY DEFINER-with-mutable-search_path` shape that constitutes the CVE-2018-1058 surface.

5. Methodology

We run each cell in a disposable Docker container (`postgres:14-alpine`, `postgres:16-alpine`; image digests recorded), against a freshly built victim, and **normalise** three preconditions to a known state so that results are comparable across PostgreSQL versions rather than confounded by default differences:

- `pub_create` — whether `PUBLIC` holds `CREATE` on `public`;
- `pin_path` — whether the trigger function's `search_path` is pinned;
- `app_owns` — whether the application role owns (and can `ALTER`) the audit function.

Separately, we record the *true stock default* per version by probing a fresh database with a fresh low-privilege role. Blinding is measured **out of band**: each attack plants a uniquely-named sentinel write, and the DBA counts whether that write reached the audit table — the ground truth of whether auditing occurred, independent of whether the attack's setup statements succeeded. All identities and addresses are synthetic (RFC 5737 documentation IP ranges [RFC5737]).

6. Results

6.1 Feasibility matrix (RQ1)

Behavior is identical on PostgreSQL 14 and 16 at the mechanism level; the difference is in what counts as "stock." Summarising the executed matrix:

Vector	Blinds a write?	Precondition it truly needs
1A plain catalog shadow (<code>evil.row_to_json</code>)	No	<code>pg_catalog</code> resolved first; app role cannot even <code>CREATE SCHEMA</code> without <code>CREATE -on-database</code>
1B exact composite overload <code>public.row_to_json(app.accounts)</code>	Yes	writable <code>public</code> and unpinned trigger <code>search_path</code>
1C replace the trigger function body	Yes	app role owns / can <code>ALTER</code> the audit function
1D bespoke <code>SECURITY DEFINER</code> helper	N/A here	no such helper beyond the built-ins in 1A/1B

The decisive, verified version nuance: on stock **PostgreSQL 14** a fresh low-privilege role **can** `CREATE` in `public` (schema ACL grants `PUBLIC USAGE+CREATE`), so 1B is reachable on defaults; on stock **PostgreSQL 15+/16** `PUBLIC` has only `USAGE`, so 1B requires an explicit re-grant or another writable schema on the function's path. **Plain catalog shadowing (1A) never blinds** — the exact-overload specificity in 1B is what beats `pg_catalog`'s search-order priority.

6.2 Reversibility (RQ2)

In 1C the attacker first captures the exact original definition (`pg_proc.prosrc` is world-readable and stored *verbatim*), replaces the body with a no-op to drop one write, then replays the captured definition byte-for-byte. The result is that `pg_get_functiondef` is identical before and after (equal MD5): **a source/object diff finds nothing**. A naive restore that merely rewrites a semantically-equivalent body does *not* match, because source is stored *verbatim* — so casual reversibility is detectable, careful reversibility is not. What still betrays a perfect restore is *independent* of the audit code: a write-versus-audit **canary** (business rows minus audit rows) shows a gap of one, and a DDL **event trigger** recorded

two `CREATE OR REPLACE FUNCTION` events. The lesson is that object-diffing the audit code is insufficient; an out-of-band write counter and a DDL event log are required.

6.3 Dormant bypass (RQ3) and attribution poisoning (RQ4)

Given the 1C ownership precondition, a `current_setting('app.maint')`-gated branch grafted into the real function skips logging only in a session that sets the flag: the gated write is not logged while a control write in the same session is. At rest the function looks legitimate. Given a broad `INSERT` grant on the audit table, the application role writes a fabricated row — a non-existent user, a synthetic documentation IP, back-dated three days — that is schema-identical to genuine rows; a reviewer using only the audit table has no field that flags it as forged. Exposure requires cross-layer correlation with an independent app-side log.

6.4 Cross-engine generalisation (MySQL family and SQL Server)

To test whether the phenomenon is PostgreSQL-specific, we reproduced the study on the MySQL family (MariaDB 10.6 and 10.11) and SQL Server (LocalDB, SQL Server 2025 / v17). The composition transfers to all three, but the exploitation surface and reversibility differ by engine — which is itself a result. The name-resolution shadowing class (1A/1B) transfers to *neither*: both lack `search_path`, so the strongest stock-default PostgreSQL vector has no analogue elsewhere.

On the **MySQL family**, whose `DEFINER`-trigger model is the analogue of `SECURITY DEFINER`, trigger replacement (1C) transfers given the `TRIGGER` privilege, but the audit trigger's `DEFINER` binds privilege: the app role cannot recreate the original `root-DEFINER` trigger without `SET USER / SUPER`, so a missing or altered trigger is an un-erasable residue — object-level tamper-evidence is *stronger* than PostgreSQL's byte-perfect restore. The dormant bypass is *harder* (a replacement app-`DEFINER` trigger cannot write the root-owned audit table, so it additionally needs an audit-`INSERT` grant).

On **SQL Server**, the "definer" analogue is ownership chaining (trigger, business table, and audit table co-owned by `dbo`). Trigger replacement (1C) transfers given `ALTER` on the table, and because the module source is stored verbatim the attacker restores it byte-for-byte — a clean object diff, like PostgreSQL. Dormancy is likewise *easy* (an `ALTER` keeps `dbo` ownership, so ownership chaining survives and no extra audit grant is needed). SQL Server also exposes a distinct, zero-residue blinding primitive absent from the other engines — `DISABLE TRIGGER` — which leaves the trigger definition unchanged (only the `is_disabled` flag flips and is restored) and, notably, raises **no** DDL event, so a DDL-trigger guard that catches `ALTER` does **not** catch it.

Attribution poisoning is identical on all three engines, and engine-level logging (`pgaudit / general_log / SQL Server Audit`) plus a hash-chained sink defend identically. Net: the self-defeating-audit threat is a three-engine class; PostgreSQL and SQL Server are the *permissive* pair (verbatim-restore reversibility, easy dormancy), while the MySQL family is the most tamper-evident owing to its `DEFINER`-privilege binding — and SQL Server uniquely adds the `DISABLE TRIGGER` primitive its own DDL defense misses.

7. Defenses (RQ5)

Every defense below was executed against the live attacks, not asserted. The defense × attack matrix:

Attack ↓ / Defense →	REVOKE CREATE on public	Pin trigger search_path	Correct ownership	DDL event- trigger guard	Hash- chained sink	Engine log (pgaudit/ log_statement)
1B overload shadow	stops	stops	—	detects*	—	logs write
1C replace fn	—	—	stops	detects	—	logs write
2 GUC dormant	—	—	stops	detects	—	logs write
3 attribution poison	—	—	—	—	exposes	corroborates actor

*The overload plant is a `CREATE FUNCTION` in `public`, not on an `audit.*` object, so an audit-scoped DDL guard must widen its filter to catch public-schema plants.

Key executed results: pinning the trigger `search_path` turns 1B from BLINDED to LOGGED even with `public` writable; correct ownership makes the app role unable to `CREATE OR REPLACE` the function, killing 1C and the dormant bypass; the DDL event trigger cannot be dropped by the app role (permission denied) and records the replacements; the hash-chained sink's verifier returns the tampered sequence number on a raw/back-dated insert (and NULL on an intact chain); and engine-level `log_statement` captures the write in the server log even while the trigger auditor is blinded — the app role can neither change the GUC (superuser-only) nor reach the log file. Only engine-level logging combined with an out-of-engine tamper-evident sink closes the whole set.

8. Discussion

What is and is not novel. The primitives — `search_path` shadowing (CVE-2018-1058) and `SECURITY DEFINER` escalation — are prior art and we claim neither. The contribution is (i) the *threat-model inversion* to a low-privilege, injection-only auditee against an unaudited auditor; (ii) the composition of reversible blinding, dormant bypass, and in-database attribution poisoning into one coherent "self-defeating audit" model; (iii) the verbatim-restore reversibility observation; and (iv) an executed defense evaluation and reproducible artifact.

Honest scoping. The strongest "works on stock defaults" claim holds only for PostgreSQL ≤ 14 ; on 15+ the `public`-schema default change makes 1B misconfiguration- gated. We state the PostgreSQL version and `public`-schema ACL for every result rather than generalising.

Limitations. We evaluate PostgreSQL (14, 16), the MySQL family (MariaDB 10.6, 10.11), and SQL Server (2025 / v17); Oracle is future work. The SQL Server experiments run on LocalDB and model the low-privilege attacker via `EXECUTE AS USER` impersonation (LocalDB is Windows-auth only), which enforces that principal's permissions but is not a separate network session. The victim is the wiki pattern; bespoke auditors may pin `search_path` or use engine logging already. We measure blinding of a single logical write, not throughput-scale evasion.

9. Responsible disclosure

This work was conducted entirely in a disposable local lab with synthetic data and no real system identified. The behaviors shown are documented PostgreSQL behavior plus operator misconfiguration, not a new engine defect; a courtesy note was sent to the PostgreSQL security team framed as such. The published SQL is scoped to the lab schema and is not a weaponised payload. The intended takeaway is the hardening checklist of Section 7.

10. Reproducibility

The complete lab — container setup, parameterised victim, attack catalogue, defenses, snapshot tooling, and the orchestrator that fills the matrices — is released as an artifact with pinned image digests and per-version evidence logs. A single driver reproduces every cell on PostgreSQL 14 and 16. Archived at Zenodo (DOI: 10.5281/zenodo.22004478) and mirrored on GitHub (https://github.com/mthamil107/DB_Audit_Research).

11. Conclusion

A SQL-injected application role, with no superuser and no OS access, can reversibly blind the widely-copied trigger-based PostgreSQL auditor and poison attribution from within the database — under preconditions that are stock defaults on PostgreSQL ≤ 14 and common misconfigurations on 15+. The individual mechanisms are known; their composition into a self-defeating audit at application privilege is not addressed by existing anti-forensics work, which assumes a privileged adversary. The practical remedy is unambiguous: do not trust an in-database, in-engine trigger log as tamper-proof against the very role it audits — pin `search_path`, revoke `CREATE` on `public`, own audit objects with a role the application can never alter, and record at the engine (pgaudit) into an out-of-engine, hash-chained, append-only sink.

References

- [PGSEC2018] PostgreSQL Global Development Group. *CVE-2018-1058*. <https://www.postgresql.org/support/security/CVE-2018-1058/>
- [PGCOMMIT2018] PostgreSQL. Documentation commit on `search_path` trust (CVE-2018-1058 fix). <https://github.com/postgres/postgres/commit/5770172cb0c9df9e6ce27c507b449557e5b45124>
- [PGWIKI2018] PostgreSQL Wiki. *A Guide to CVE-2018-1058: Protect Your Search Path*. https://wiki.postgresql.org/wiki/A_Guide_to_CVE-2018-1058
- [CYBERTEC] Cybertec. *Abusing SECURITY DEFINER functions*. <https://www.cybertec-postgresql.com/en/abusing-security-definer-functions/>
- [CYBERTEC-ROW] Cybertec. *Row change auditing options for PostgreSQL*. <https://www.cybertec-postgresql.com/en/row-change-auditing-options-for-postgresql/>
- [MemTraceDB] *MemTraceDB*. arXiv:2509.05891. <https://arxiv.org/pdf/2509.05891>
- [CROSBY2009] S. A. Crosby and D. S. Wallach. *Efficient Data Structures for Tamper-Evident Logging*. USENIX Security 2009. https://static.usenix.org/event/sec09/tech/full_papers/crosby.pdf
- [SCHNEIER1998] B. Schneier and J. Kelsey. *Cryptographic Support for Secure Logs on Untrusted Machines*. USENIX Security 1998.
- [RFC6962] B. Laurie, A. Langley, E. Kasper. *Certificate Transparency*. RFC 6962.
- [RFC5737] J. Arkko, M. Cotton, L. Vegoda. *IPv4 Address Blocks Reserved for Documentation*. RFC 5737.
- [PGAUDIT] pgAudit: PostgreSQL Audit Extension. <https://github.com/pgaudit/pgaudit>